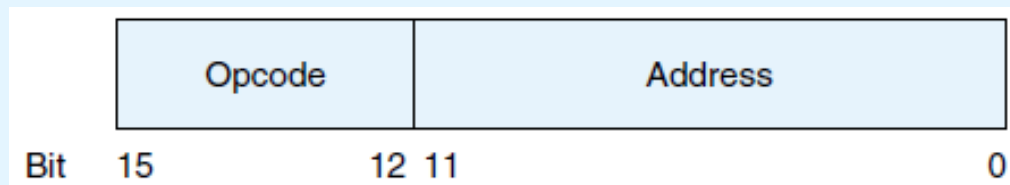


“The Essentials of Computer Organization and Architecture”

Unidade 3 – Exercícios

MD1 Considerando o conjunto de instruções da arquitetura MARIE, escreva em assembly as representações correspondentes dos seguintes valores binários:

- i) 0010000000000111
- ii) 1001000000001011
- iii) 0011000000001001



Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT1 a) assemble o seguinte programa, e
b) faça a traçagem das 4 primeiras instruções:

Endereço	Etiqueta	Instrução
100		Load A
101		Add One
102		Jump S1
103	S2,	Add One
104		Store A
105		Halt
106	S1,	Add A
107		Jump S2
108	A,	HEX 0023
109	One,	HEX 0001

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT2 Resolva de novo o exercício MAT1 assumindo que o programa ocupa a memória a partir do endereço 200_{16} .

MAT3 a) assemble o seguinte programa, e
b) faça a traçagem das 4 primeiras instruções:

ORG 100

```

If,      100 Load      X      /Load the first value
          101 Subt      Y      /Subtract the value of Y, store result in AC
          102 Skipcond  400    /If AC=0 (X=Y), skip the next instruction
                                   /Note: This is Skipcond 01 in the book
          103 Jump      Else    /Jump to Else part if AC is not equal to 0
Then,    104 Load      X      /Reload X so it can be doubled
          105 Add       X      /Double X
          106 Store     X      /Store the new v
          107 Jump      Endif   /Skip over the f
Else,    108 Load      Y      /Start the else
          109 Subt      X      /Subtract X from
          10A Store     Y      /Store Y-X in Y
Endif,   10B Halt       /Terminate progr
X,       10C Dec        12     /Assume these va
Y,       10D Dec        20
  
```

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT4 Resolva de novo o exercício MAT3 assumindo que o programa ocupa a memória a partir do endereço 400_{16} .

MAT5 a) assemble o seguinte programa, e
b) faça a traçagem das 2 primeiras instruções:

ORG 100

```

100 Load X /Load the first number t
101 Store Temp /Use Temp as a parameter
102 JnS Subr /Store return address, i
103 Store X /Store first number, dou
104 Load Y /Load the second number
105 Store Temp /Use Temp as a parameter
106 JnS Subr /Store return address, i
107 Store Y /Store second number, dc
108 Halt /End program
X, 109 Dec 20
Y, 10A Dec 48
Temp, 10B Dec 0
Subr, 10C Hex 0 /Store return address he
10D Clear /Clear AC as it was mod
10E Load Temp /Actual subroutine to do
10F Add Temp /AC now hold double the
110 JumpI Subr /Return to calling code

```

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

Instruction Number (hex)	Instruction
0	JnS X
A	Clear
B	AddI X
C	JumpI X
D	LoadI X
E	StoreI X

MAT6 Considere de novo o programa do exercício MAT5, mas assuma que começa no endereço 300_{16} .

1. Assemble o programa nas novas condições indicadas e apresente:
 - a) A tabela de símbolos que resulta da primeira passagem (preencha a tabela segundo a ordem pela qual o processo de assemblagem encontra os símbolos no código fonte);
 - b) A listagem com os *opcodes* (em formato hexadecimal) e símbolos que resulta da primeira fase da assemblagem;
 - c) A listagem com o resultado final da assemblagem, em formato hexadecimal.

MAT6 Considere de novo o programa do exercício MAT5, mas assuma que começa no endereço 300_{16} .

2. Com base na resposta à questão 1., faça a traçagem das primeiras quatro instruções executadas pelo programa e apresente, na tabela abaixo, cada instrução (pela ordem de execução) e o valor final dos registros MAR, MBR, PC, IR e AC imediatamente após a execução dessa instrução.

	Instrução 1:	Instrução 2:	Instrução 3:	Instrução 4:
MAR				
MBR				
PC				
IR				
AC				

MC1 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

```
int X=20, Y=10;  
if (X > 1) {  
    Y = X + X;  
    X = 0;  
}  
Y = Y + 1;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC2 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

```
int X = 1;
while (X < 10)
    X = X + 1;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC2 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

Nota:

```
int X = 1;
while (X < 10)
    X = X + 1;
```

<=>

```
int X = 1;
Loop: if (X < 10) {
        X = X + 1;
        goto Loop;
    }
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC3 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

```
int X = 1;
do {
    X = X + 1;
} while (X < 10)
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC3 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

Nota:

```
int X = 1;
do {
    X = X + 1;
} while (X < 10)
```

<=>

```
int X = 1;
Loop: X = X + 1;
      if (X < 10)
          goto Loop;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC4 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

```
int X = 1, Sum = 0;
while (X < 11) {
    Sum = Sum + X;
    X = X + 1;
}
printf("%d", Sum);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC5 Resolva de novo o exercício MC4, mas considerando a condição $X \leq 11$ em vez de $X < 11$.

```
int X = 1, Sum = 0;
while (X <= 11) {
    Sum = Sum + X;
    X = X + 1;
}
printf("%d", Sum);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC6 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

```
int i, a[5]={'0','2','4','6','8'};

for(i=0; i<5; i++)
    printf("%c", a[i]);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

MC6 Escreva o seguinte fragmento de código C em assembly da arquitetura MARIE:

Nota:

```
int i, a[5]={'0','2','4','6','8'};
for(i=0; i<5; i++)
    printf("%c", a[i]);
```

<=>

```
int i=0;
int a[5]={'0','2','4','6','8'};
Loop: if (i < 5) {
    printf("%c", a[i]);
    i = i + 1;
    goto Loop;
}
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

MC7 O seguinte código tem uma rotina principal (main) que invoca uma pseudo-subrotina (sub1) para duplicar o valor “global” X. Quais os potenciais problemas do seguinte código?

```
Main,    Load X
          Jump Sub1
Sret,    Store X
...
Sub1,    Add X
          Jump Sret
```

MC8 Escreva um programa, em assembly MARIE, com uma subrotina para subtrair dois números, que será invocada duas vezes, de forma que $E = A - B$ e $F = C - D$.

Versão em C com uma invocação: sequência de execução

```
int main()
{
    int A, B, E;
    scanf("%d%d", &A, &B);

    E = subr(A, B);

    printf("%d", E);
    return 0;
}
```

x e **y** são os parâmetros **formais** da função **subr** (i.e., fazem parte da assinatura / contrato / definição / declaração da função).

```
int subr ( int x, int y )
{
    int res;

    res = x - y;
    return res;
}
```

A e **B** são os parâmetros **atuais** da função **subr** (i.e., são os efetivamente passados à função neste caso particular)

MC8 Escreva um programa, em assembly MARIE, com uma subrotina para subtrair dois números, que será invocada duas vezes, de forma que $E = A - B$ e $F = C - D$.

Versão em C com uma invocação: passagem de parâmetros

```
int main()
```

```
{  
    int A, B, E;  
    scanf ("%d%d", &A, &B) ;
```

```
    E = subr (A, B) ;
```

```
    printf ("%d", E) ;
```

```
    return 0;
```

```
}
```

```
int subr( int x, int y )
```

```
{  
    int res;
```

```
    res = x - y;
```

```
    return res;
```

```
}
```

O valor de **A** é copiado para a variável **x** da função **subr**.

O valor de **B** é copiado para a variável **y** da função **subr**.

MC8 Escreva um programa, em assembly MARIE, com uma subrotina para subtrair dois números, que será invocada duas vezes, de forma que $E = A - B$ e $F = C - D$.

Versão em C com uma invocação: retorno de resultado

```
int main()
{
    int A, B, E;
    scanf("%d%d", &A, &B);

    E ← subr(A, B);

    printf("%d", E);
    return 0;
}
```

```
int subr(int x, int y)
{
    int res;

    res = x - y;
    return res;
}
```

O valor de **res** é copiado para a variável **E** da função **main**.

MC8 Escreva um programa, em assembly MARIE, com uma subrotina para subtrair dois números, que será invocada duas vezes, de forma que $E=A-B$ e $F=C-D$.

Versão em C completa:

```
int Subr (int X, int Y)
{
    int Res;
    Res = X - Y;
    return(Res);
}
Main()
{
    int A=8,B=4,C=10,D=2,E,F;
    E=Subr(A,B);printf("%d",E);
    F=Subr(C,D);printf("%d",F);
}
```

MC9 Escreva um programa que permita multiplicar dois números inteiros positivos recorrendo a um ciclo de adições sucessivas. Exemplo: $3 \times 6 = 3 + 3 + 3 + 3 + 3 + 3$

Sugestão: comece por escrever o programa em C e a seguir “transcreva-o” para assembly MARIE.

MC10 Escreva um programa, em assembly MARIE, que utiliza uma subrotina para obter o resultado da seguinte expressão: $G = (E = A \times B) + (F = C \times D)$

Sugestão: reutilize o código do exercício MC9 na implementação da subrotina

MC11 Codifique em assembly MARIE as seguintes rotinas, e programas simples que as invoquem:

- a) Uma rotina “Min” para o cálculo do mínimo entre 2 inteiros
- b) Uma rotina “Max” para o cálculo do máximo entre 2 inteiros
- c) Uma rotina “Sym” para o cálculo do simétrico de 1 inteiro
- d) Uma rotina “Div” para o cálculo do quociente da divisão inteira entre 2 inteiros
- e) Uma rotina “Mod” para o cálculo do resto da divisão inteira entre 2 inteiros
- f) Uma rotina “Prime” que verifique se um inteiro é primo

Nota: para prevenir colisão de nomes, cada variável tem como prefixo o nome da sua rotina (exemplo: Min_X é a variável X da rotina Min).

MC12 Codifique em assembly MARIE as seguintes rotinas, e programas simples que as invoquem:

- a) Uma rotina “MinArray” para o cálculo do mínimo de um array de inteiros
- b) Uma rotina “MaxArray” para o cálculo do máximo de um array de inteiros
- c) Uma rotina “ShowSymArray” que, dado um array de inteiros, apresente no ecrã o simétrico de cada elemento do array
- d) Uma rotina “TestPrimeArray” que percorra um array de inteiros e, para cada elemento, apresente 0 ou 1 no ecrã, consoante o elemento em análise é ou não primo

MC13 Codifique em assembly MARIE as seguintes rotinas, e programas simples que as invoquem:

- a) Uma rotina “MakeSymArray” que, dado um array de inteiros, produza um array com os simétricos dos inteiros do primeiro array.
- b) Uma rotina “MakelsPrimeArray” que, dado um array de inteiros, verifique se os mesmos são primos, produzindo um array com o resultado dessa verificação para cada inteiro (1 = True, 0 = False).

MATC1 – Exercício de Consolidação

(Assemblagem, Traçagem e Codificação)

1. Considere o seguinte programa codificado em assembly MARIE:

```
ORG 100
Loop,   Load Counter
        Subt Num
        Skipcond 800
        Jump LoopBody
        Jump LoopEnd
LoopBody, Load Counter
        Add One
        Store Counter
        Jump Loop
LoopEnd, Halt
Counter, Dec 2
Num,    Dec 10
One,    Dec 1
```

Opcode	Instruction
1	<u>Load</u> X
2	<u>Store</u> X
3	<u>Add</u> X
4	<u>Subt</u> X
5	Input
6	Output
7	<u>Halt</u>
8	<u>Skipcond</u> X X=000: <u>AC</u> <0? X=400: <u>AC</u> ==0? X=800: <u>AC</u> >0?

Instruction Set
do MARIE

Opcode	Instruction
9	<u>Jump</u> X
0	<u>Jns</u> X
A	<u>Clear</u>
B	<u>AddI</u> X
C	<u>JumpI</u> X
D	<u>LoadI</u> X
E	<u>StoreI</u> X

1.1 Assemble o programa e preencha as tabelas seguintes
(nota: todas as tabelas têm já o número de linhas correto)

MATC1 – Exercício de Consolidação

(Assemblagem, Traçagem e Codificação)

1.1.a Tabela de Símbolos que resulta da primeira passagem
(preencha a tabela segundo a ordem pela qual o processo de assemblagem encontra os símbolos no código fonte; apresente os endereços em hexadecimal):

Tabela de Símbolos

Símbolo	Endereço

MATC1 – Exercício de Consolidação

(Assemblagem, Traçagem e Codificação)

1.1.b Listagem com os endereços das posições de memória usadas, e seu conteúdo (resultado final da assemblagem), em hexadecimal:

Endereço	Conteúdo

MATC1 – Exercício de Consolidação

(Assemblagem, Traçagem e Codificação)

2. Com base na resposta à questão 1.1.b), faça a traçagem das primeiras cinco instruções executadas pelo programa. Depois, na tabela abaixo apresente cada instrução (Inst1 a Inst5, pela sua ordem de execução) e o valor final (em hexadecimal) dos registros MAR, MBR, PC, IR e AC logo após a execução dessa instrução:

Instruction / CPU Register	Inst1:	Inst2:	Inst3:	Inst4:	Inst5:
MAR:					
MBR:					
PC:					
IR:					
AC:					

MATC1 – Exercício de Consolidação

(Assemblagem, Traçagem e Codificação)

3. Os números de Lucas são gerados pela fórmula $L_n = L_{n-2} + L_{n-1}$, com $L_0 = 2$ e $L_1 = 1$, e com cada termo seguinte igual à soma dos dois termos anteriores. Por exemplo, os primeiros 5 números desta sequência são: 2, 1, 3 (=1+2), 4 (=3+1), 7 (=4+3). O seguinte programa em C contém a função **Lucas**, que retorna o N-ésimo número de Lucas, e uma função **main**, que tira partido dela (nota: pode assumir que é sempre fornecido um valor de N não-negativo).

```
int main()
{
    int N, L;

    scanf("%d", &N);

    if (N < 2)
        L = 2 - N;
    else
        L = Lucas(N);

    printf("%d", L);
}
```

```
int Lucas(int Num)
{
    int Number0, Number1, Counter, Current;
    Number0 = 2; Number1 = 1;

    for (Counter = 2; Counter <= Num; Counter++) {
        Current = Number1 + Number0;
        Number0 = Number1;
        Number1 = Current;
    }

    return Current;
}
```

MATC1 – Exercício de Consolidação (Assemblagem, Traçagem e Codificação)

3.a) Apresente o código assembly MARIE da função **main**.

```
int main()  
{  
    int N, L;  
  
    scanf("%d", &N);  
  
    if (N < 2)  
        L = 2 - N;  
    else  
        L = Lucas(N);  
  
    printf("%d", L);  
}  
  
int Lucas(int Num)  
{  
    int Number0, Number1, Counter, Current;  
    Number0 = 2; Number1 = 1;  
  
    for (Counter = 2; Counter <= Num; Counter++) {  
        Current = Number1 + Number0;  
        Number0 = Number1;  
        Number1 = Current;  
    }  
  
    return Current;  
}
```

MATC1 – Exercício de Consolidação

(Assemblagem, Traçagem e Codificação)

3.b) Apresente o código assembly MARIE da função **Lucas**.

```
int main()
{
    int N, L;

    scanf("%d", &N);

    if (N < 2)
        L = 2 - N;
    else
        L = Lucas(N);

    printf("%d", L);
}
```

```
int Lucas(int Num)
{
    int Number0, Number1, Counter, Current;
    Number0 = 2; Number1 = 1;

    for (Counter = 2; Counter <= Num; Counter++) {
        Current = Number1 + Number0;
        Number0 = Number1;
        Number1 = Current;
    }

    return Current;
}
```